

中山大學

SUN YAT-SEN UNIVERSITY

多模态大模型实验报告

Web-MMAgent：网页多模态 Agent （界面理解 + 自动操作）

组 号 第八组

姓 名 王胜伟、王一澄、吴乐超

学 号 23336228、23336233、23336250

学 院 计算机学院

专 业 计算机科学与技术

2026 年 6 月 22 日

摘　要

本项目选择课程题目 7 “网页多模态 Agent（界面理解与自动操作）”，实现了一个面向真实网页任务的可复现 Web Agent 系统 Web-MMAgent。系统以自然语言任务和起始网页为输入，通过 Playwright 控制浏览器，构建包含网页截图、Set-of-Marks 标注图、DOM 候选元素、Accessibility Tree、页面文本片段和动作历史的混合观察空间（observation space）；模型侧采用 ReAct 风格单步决策，输出受约束的结构化 JSON 动作；执行侧在每一步进行动作合法性校验、安全拦截、候选元素定位、进度检查和完成验证，并保存完整 trace。项目进一步实现了 YAML 任务管理、批量运行、指标统计、确定性答案校验、开放任务字段覆盖校验以及 HTML 轨迹回放。

相较于课程最低要求中“选择 1–2 个真实网站、每站至少 3 个任务、统计成功率并提供截图序列”的标准，本项目扩展为三类真实网站场景共 9 个任务：CSRankings、Bilibili 与广州旅游规划，每类均包含 easy、medium、hard 难度。评测体系除任务成功率外，还记录平均步数、平均用时、动作 JSON 合法率、grounding error rate、纠错重试成功率和失败类型分布。我们首先使用 DeepSeek V4 Pro 对 9 个任务各运行 10 次，共 90 次；统计以项目随附的《实验报告》为准，其中 76 次成功，综合成功率为 84.44%，平均 12.7611 步、439.5247 秒，失败类型为 planning 13 次、perception 1 次。随后使用 GPT-5.5 对 9 个任务各运行一次：主流程均正常结束，harness success 为 9/9，平均 10.89 步、721.10 秒。最后选取 10 个来自 OpenAI、Google、Anthropic、Qwen、DeepSeek、Kimi、GLM 和豆包系列的模型，对 9 个任务构成的 90 个模型–任务组合进行横向测试。报告将有效样本的能力差异与 API 超时、内容安全拦截、图文上下文超限等无效样本分开呈现，避免将服务层问题误判为模型能力差异。

当前报告给出系统架构、观察空间与动作空间设计、任务定义、评测口径、DeepSeek V4 Pro 重复评测、GPT-5.5 基线、十模型横向评测结果、失败类型分析及可视化轨迹来源。实验表明，确定性信息检索任务可以借助脚本严格判分，而开放式网页任务必须结合字段覆盖、访问证据与人工或 LLM-as-judge 复核；长链任务的主要挑战在于计划分解、证据管理和真实网页环境稳定性。因此，本项目不仅展示了一个可运行的多模态网页操作系统，也总结了真实网页环境下 Web Agent 评测需要同时报告自动指标、严格校验与 trace 证据的必要性。

关键词：网页 Agent；多模态大模型；Playwright；Set-of-Marks；ReAct；批量评测

目录

1 引言 **1**

1.1 任务背景 1
1.2 问题定义 2
1.3 团队分工 2
1.4 项目贡献 3

2 相关工作 **3**

2.1 真实网页任务与评测环境 3
2.2 界面理解与 Grounding 4
2.3 推理-行动闭环 4
2.4 GUI Agent 的扩展启发 4
2.5 本项目与已有工作的区别 5

3 系统设计与实现 **5**

3.1 总体架构 5
3.2 观察空间 6
3.3 候选元素与 Set-of-Marks 7
3.4 动作空间与安全校验 7
3.5 ReAct Prompt 与单步决策 8
3.6 Progress Checker 与 Final Verifier 9
3.7 Trace 格式与可视化 9

4 任务设计与评测方案 **10**

4.1 任务库设计原则 10
4.2 难度划分 11
4.3 评测指标与判分口径 12
4.4 第一轮评测：DeepSeek V4 Pro 九任务重复评测 13
4.5 GPT-5.5 九任务基线评测 14
4.6 十模型横向评测：先看有效样本 16
4.7 十模型评测中的无效样本与风险 17
4.8 数据来源与可视化轨迹 18

5 错误分析、局限与展望 **18**

5.1 从有效样本看失败模式 18
5.2 失败类型与改进方向 19
5.3 当前局限 19
5.4 后续工作 20
5.5 总结 20

1 引言

1.1 任务背景

多模态大模型正在从“看图回答问题”走向“理解界面并执行操作”。网页是最典型也最复杂的图形用户界面之一：页面同时包含文本、按钮、表单、弹窗、图片、布局层级和动态加载内容；用户任务又往往不是单步问答，而是需要跨页面搜索、筛选、比较、记录证据并生成最终答案。因此，网页 Agent 的难点不只是让模型读懂页面，而是让模型在真实浏览器环境中稳定完成“观察、规划、操作、检查、验证”的闭环。

本课程题目 7 要求设计网页多模态 Agent：基于网页截图/结构信息和自然语言指令，生成点击、输入、滚动等浏览器操作并完成具体任务。课程给出的最低验收标准包括：选择 1–2 个真实网站，每个网站至少 3 个任务场景；每个场景至少 10 次独立测试，总测试不少于 60 次；报告任务成功率；分类记录识别失败、规划失败和执行失败；提供截图序列与动作序列等可视化操作轨迹。

我们实现的 Web-MMAgent 将该要求进一步工程化：系统不只演示若干成功案例，而是把网页任务抽象为可配置的 YAML episode，把每一步观察、模型响应、结构化动作、执行结果、进展检查和完成验证都写入 trace，从而支持批量评测、错误分析和轨迹回放。

表 1 课程最低要求与本项目实现目标对照

维度	课程最低要求	本项目实现与扩展
网站与任务	1–2 个真实网站，每站至少 3 个任务	3 类真实网站/场景，共 9 个任务，均按 easy/medium/hard 分层
运行方式	浏览器自动化演示	Playwright/CDP 浏览器环境，结构化动作执行，支持 headless 与可视化运行
观察信息	网页截图/结构信息	原始截图、SoM 标注截图、DOM 候选元素、Accessibility Tree、页面文本、动作历史
评测指标	至少成功率	成功率、平均步数、平均用时、动作 JSON 合法率、定位错误率、纠错成功率、失败类型分布
轨迹材料	截图序列 + 动作序列	每步保存 screenshot、marked screenshot、candidates、prompt、raw response、parsed action、execution result、progress check、final verification
安全约束	未明确展开	拦截登录、支付、购买、删除、敏感信息提交、验证码绕过等高风险行为

1.2 问题定义

一个网页任务 episode 由五部分组成：

1. 起始地址 u_0，例如 CSRankings 的过滤页面或 B 站搜索结果页；
2. 自然语言任务指令 I，描述用户希望获得的最终结果；
3. 最大步数 T，防止模型陷入无限循环；
4. 成功标准 C，可以是确定性答案、字段覆盖、页面证据数量或开放式人工复核标准；
5. 禁止动作集合 F，例如登录、支付、下单、删除、提交敏感信息等。

1.3 团队分工

本项目由三名成员协作完成，分工围绕系统架构、任务设计、工程实现、评测分析与展示材料展开，如表 2 所示。

表 2 团队成员分工

成员	负责方向	贡献说明
王胜伟	项目架构、基线与评测	负责 Agent 架构、评测系统和实验流程设计；搭建项目基线，实现 Playwright 浏览器环境、结构化动作、trace 保存和批量评测；设计并实现 CSRankings easy/medium/hard 三个任务，完成大部分评测、结果整理、错误分析、文献与项目调研、开题报告及最终报告主要撰写。
王一澄	工程化完善与广州任务	对项目进行整体工程化改造，完善多模态输入、真实浏览器后端、长任务 runner、checklist、Progress Checker、Final Verifier、WebUI 和 HTML 报告；扩展 raw screenshot、SoM、Accessibility Tree 与 DOM candidates 联合 observation，以及 Playwright/CDP/WebAccess 后端和 navigate/search/ask_user 动作；设计并实现广州旅游 easy/medium/hard 任务，负责开题与期末 PPT 并完成两次展示。
吴乐超	Bilibili 任务与实验协作	负责 Bilibili easy/medium/hard 三个任务的设计与实现；参与项目代码实现、评测实验、结果核验、数据分析和实验报告撰写。

在第 t 步，Agent 从浏览器获得观察 o_t，模型输出结构化动作 a_t，执行器验证并执行动作，浏览器转移到新状态 s_{t+1}。若模型输出 `answer` 或 `done`，系统调用 Final Verifier 检查最终答案是否有页面证据支持；若达到最大步数或被判定为卡住，系统生成失败/部分完成结果并保留 trace。整个过程可以概括为：

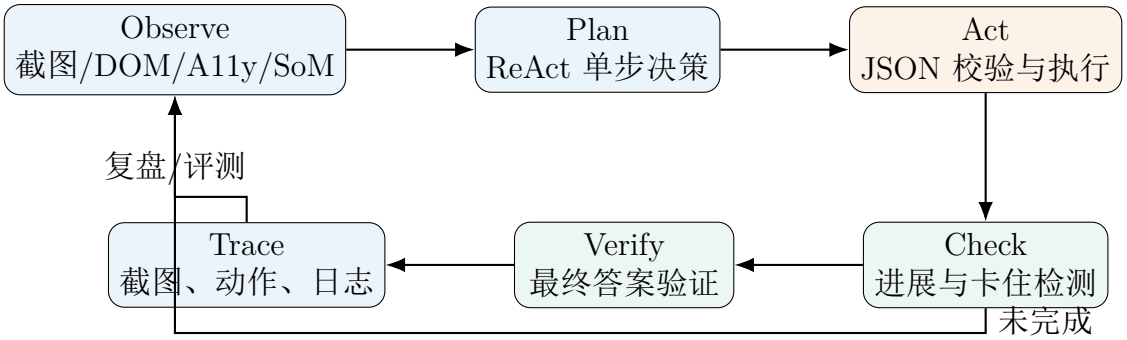


图 1 Web-MMAgent 的闭环问题定义

1.4 项目贡献

本项目的主要贡献如下。

1. **实现了最小可运行到批量评测的完整闭环。**从 Phase 1 的 Playwright 浏览器环境、候选元素提取和 mock agent，到 Phase 5 的 YAML 任务管理、批量 runner、指标统计与 HTML trace report，形成了端到端可运行系统。
2. **设计了混合观察空间。**对视觉模型提供原始截图和 SoM 标注图，对纯语言模型提供候选元素、页面文本和 Accessibility Tree，使不同能力模型可以在同一动作接口下比较。
3. **将动作空间约束为结构化 JSON。**模型不能执行任意代码，也不能直接发出鼠标坐标；所有 click/type 都必须引用当前候选元素的 `target_id`，并通过安全校验。
4. **构造了 9 个分层真实网页任务。** CSRankings 任务适合确定性答案评测；Bilibili 和广州旅游规划任务更开放，用来考察跨页面信息收集、证据保存和长答案生成能力。
5. **提供可复盘的评测材料。**每一次运行都保存截图序列、标注图、候选元素、prompt、模型原始输出、解析后动作和执行结果；评测报告可以定位失败发生在识别、规划、执行、验证还是网站环境环节。

2 相关工作

2.1 真实网页任务与评测环境

网页 Agent 的研究首先需要可复现的任务环境。Mind2Web [1] 收集跨 137 个网站的真实网页操作轨迹，强调真实 HTML 往往很长，模型需要先筛选候选元素再预测动作。WebArena [2] 构建了可自托管的真实感网页环境，通过端到端任务成功率评价 Agent。VisualWebArena [3] 在此基础上加入视觉密集任务，说明仅依赖 DOM 文本难以理解图标、图片、空间布局和视觉状态。

WebVoyager [4] 更接近本项目的目标：它直接面向真实在线网站，使用网页截图和 Accessibility Tree 构建观察，并用大模型执行连续网页操作。BrowserGym/AgentLab [5]

则提供标准化 WebAgent 实验接口，对 observation space、action space、任务初始化和批量实验管理有很强参考价值。我们的系统没有直接复用这些 benchmark，而是吸收其任务组织方式：每个任务显式定义起始 URL、自然语言目标、最大步数、成功标准、禁止动作和 trace 输出。

2.2 界面理解与 Grounding

网页操作的关键瓶颈是 grounding：模型可能知道“应该点击搜索框”或“应该展开某个学校”，但无法稳定定位到页面上的准确元素。SeeAct [6] 研究了多模态模型作为网页 Agent 时的 grounding 问题，提出将视觉理解与候选元素定位结合。Set-of-Marks [7] 通过在图像上叠加编号标记，把“点击某个视觉区域”转化为“选择某个编号”，降低了直接坐标预测的难度。

Web-MMAgent 采用类似思想：DOM 提取器从页面中筛选按钮、链接、输入框、文本框、下拉框以及常见 ARIA role 元素，记录每个候选元素的编号、标签、角色、文本、选择器、bbox、可见性和可用性；SoM 模块再把可见且面积合理的元素绘制成编号框。模型最终只能通过 `target_id` 引用候选元素，执行器会在真实点击前再次确认该编号存在、可见且可用。

2.3 推理-行动闭环

ReAct [8] 将推理和行动交替组织，使模型在每一步先根据观察产生简短 thought，再选择一个工具动作。这种方式适合网页任务，因为网页状态会在每次点击、输入、滚动后改变，模型不能一次性可靠预测完整动作序列。我们采用“单步 ReAct + 外部循环”的方式：模型每次只输出一个 JSON 动作，循环逻辑、浏览器执行、失败检测和 trace 保存由宿主程序控制。

与端到端坐标 Agent 相比，这种设计更工程化，也更容易调试。若模型输出非法 JSON，Agent 层会保存 `raw_response.txt`，自动重试一次；若仍失败，则退化为安全 `wait` 动作。若模型输出了不存在的 `target_id`，执行器会拒绝执行并记录 grounding error，而不是盲目点击。

2.4 GUI Agent 的扩展启发

OSWorld [9]、Windows Agent Arena [10] 和 AndroidWorld [11] 把 Agent 从网页扩展到桌面和移动系统，强调真实环境、任务初始化、状态检查和轨迹级错误分析。UI-TARS [12]、OmniParser [13]、SeeClick [14] 等工作进一步说明，界面元素解析和视觉 grounding 是 GUI Agent 的核心能力。虽然本项目聚焦浏览器网页，不引入 OSWorld 或 PyAutoGUI，但借鉴了这些工作的两个原则：一是操作必须在真实环境中执行验证，二是失败必须能追溯到具体观察和动作。

表 3 代表性相关工作与本项目对应模块

方向	代表工作	对本项目的启发
网页任务数据集	Mind2Web [1]、WebLINX [15]	真实网站任务、候选元素筛选、动作历史建模
网页评测环境	WebArena、VisualWebArena	端到端成功率、视觉网页任务、执行式验证
真实网页 Agent	WebVoyager、SeeAct	截图 + A11y + VLM 的操作闭环，元素 grounding
视觉标注增强	Set-of-Marks	将视觉区域转化为编号选择，降低坐标定位难度
推理行动框架	ReAct	单步观察、思考、行动、反馈的多步任务循环
实验框架	BrowserGym/AgentLab	标准化 observation/action space、批量实验和报告输出
通用 GUI Agent	OSWorld、UI-TARS、OmniParser	轨迹级错误分析、屏幕解析、状态检查

2.5 本项目与已有工作的区别

本项目不是构造一个新的大型 benchmark，也不训练新的 VLM；目标是在课程项目规模内实现一个可运行、可评测、可展示的网页多模态 Agent 原型。因此我们更强调工程闭环和可复盘性：真实网页任务来自 CSRankings、Bilibili 和旅行规划；模型通过 OpenAI-compatible API、DeepSeek 或 Gemini 代理接入；浏览器执行由 Playwright 负责；评测脚本把确定性任务和开放任务分开处理。这样的设计使系统既能完成课程验收，也能作为后续扩展到更多模型和更多网站的实验平台。

3 系统设计与实现

3.1 总体架构

Web-MMAgent 的代码按浏览器执行、感知标注、模型调用、Agent 决策、评测与可视化回放分层组织。核心运行入口覆盖单任务、批量评测和多模型套件。单任务运行时，系统从 YAML 任务或命令行读取起始 URL、任务指令、最大步数、模型 provider 和 observation mode；随后启动浏览器，进入 Observe–Plan–Act–Check–Verify 循环。

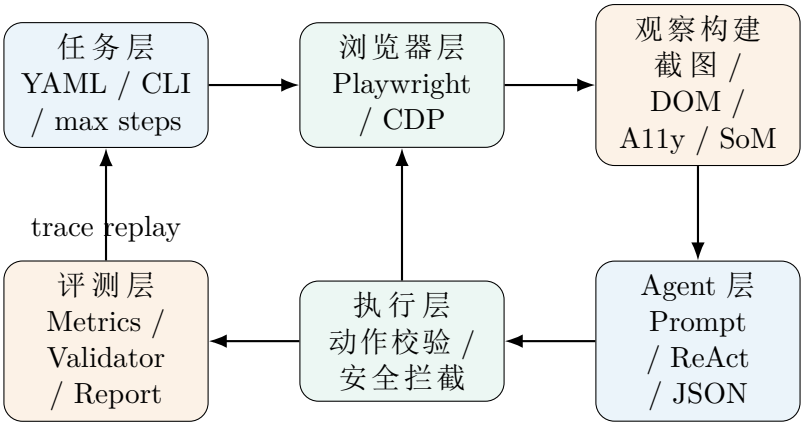


图 2 Web-MMAgent 模块架构

表 4 核心目录与职责

目录/文件	职责
`browser/`	Playwright/CDP 浏览器环境、DOM 候选元素提取、动作 schema 与执行校验
`perception/`	Set-of-Marks 标注图生成、候选元素过滤
`models/`	OpenAI-compatible、DeepSeek、Gemini 客户端，统一 chat completion 接口
`agents/`	ReAct prompt、单步 Agent、Progress Checker、Final Verifier、任务 checklist、最终 HTML 报告生成
`eval/`	单任务 runner、批量 runner、指标统计、validator、summary/report 生成
`scripts/`	demo、SoM demo、单步 Agent、多步任务、批量评测和模型 suite 命令行入口
`tasks/`	最终保留的 9 个任务 YAML：CSRankings、Bilibili、广州旅游规划
`webui/`	Trace Viewer / Workbench，可查看运行状态与轨迹

3.2 观察空间

观察空间决定模型“看见”什么。真实网页任务中，单一模态很容易失效：截图能表示视觉布局，却缺少可执行选择器；DOM 有元素结构，却不一定反映视觉遮挡和用户实际看到的内容；Accessibility Tree 有语义标签，但不总是完整；页面文本适合阅读任务，却难以定位点击对象。因此系统采用混合观察。

表 5 观察空间组成

观察项	保存文件/字段	用途
原始截图	`screenshot.png`	保留页面真实视觉状态、图片、布局、遮挡和弹窗
SoM 标注图	`marked_screenshot.png`	在可见候选元素上绘制编号框，帮助视觉模型选择 `target_id`
候选元素列表	`candidates.json`	提供 `element_id`、tag、role、text、selector、bbox、visible、enabled
Accessibility Tree	A11y tree	提供角色、标签、标题、对话框等语义结构
页面文本片段	`page_text.txt` / prompt excerpt	用于阅读、证据抽取、跨页面记忆
动作历史	prompt 中的 compact history	记录最近动作、URL、执行结果、进展信号和证据摘要
任务 checklist	`checklist.json`	将开放任务拆成可跟踪的证据项，减少长任务遗忘

系统支持三种 observation mode：`text` 只发送候选元素、页面文本和 A11y 信息，适合无视觉能力的语言模型；`vision` 额外把原始截图和 SoM 图作为图像输入，适合 Gemini、GPT、Claude、Qwen-VL 等视觉模型；`auto` 根据 provider 自动选择。这样可以在同一任务上比较纯文本模型与视觉语言模型的差异。

3.3 候选元素与 Set-of-Marks

DOM 提取器至少覆盖 button、a、input、textarea、select，以及 role 为 button、link、textbox 的元素和可编辑元素。每个候选元素被赋予稳定的 `element_id`，并保存页面 bbox。SoM 绘制阶段会过滤不可见元素、面积过小元素和屏幕外元素，然后用 Pillow 绘制轻量编号框。这样模型不需要输出坐标，只需选择编号；执行器再根据 selector 滚动到元素并点击/输入。

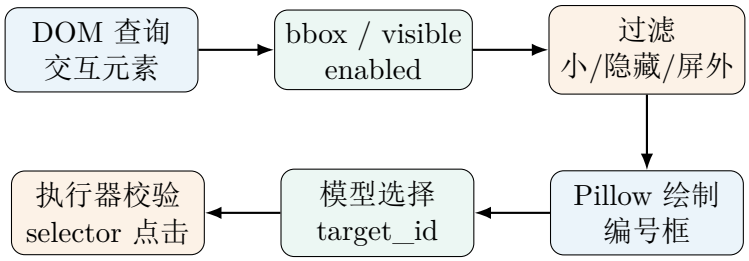


图 3 候选元素与 Set-of-Marks grounding 流程

3.4 动作空间与安全校验

动作空间采用结构化 JSON，模型只能输出下列动作：`click`、`type`、`scroll`、`press`、`back`、`wait`、`answer`、`done`、`ask_user`、`navigate`、`search`。其中 `click` 和 `type` 必须携带合法 `target_id`；其他动作强制将 `target_id` 置空。

表 6 动作空间与执行约束

动作	关键字段	执行约束
`click`	`target_id`	目标必须存在、可见、可用；禁止点击登录、支付、购买、删除、提交等高风险控件
`type`	`target_id`, `text`, `key`	文本不能为空；禁止向密码、信用卡、地址、真实联系方式等敏感输入框填入信息
`scroll`	`direction`	仅允许 up/down；用于寻找更多候选或页面文本
`press`	`key`	只执行键盘按键，例如 Enter 或 Escape
`navigate`	`text`	必须是完整 http/https/file URL，避免模型拼接危险或不完整地址
`search`	`text`	通过搜索引擎查询公开信息，不替代指定网站步骤
`answer/done`	`text`	触发 Final Verifier；答案必须尽量由当前页面或历史证据支持
`ask_user`	`text`	遇到验证码、登录墙、模糊选择或安全风险时暂停询问

动作校验由 `browser/actions.py` 中的 Pydantic schema 和规则函数完成。高风险点击词包括 delete、remove、purchase、buy、pay、checkout、order、subscribe、login、sign in、submit 等；敏感输入词包括 password、credit card、cvv、address、ssn 等。该设计保证即使模型产生危险计划，执行器也不会直接执行。

3.5 ReAct Prompt 与单步决策

Agent prompt 明确要求模型“只返回一个 JSON 对象，不要 markdown，不要代码”。输入给模型的信息包括任务指令、当前 URL、截图路径或图像输入、候选元素列表、页面文本、Accessibility Tree、人类提示、任务 checklist 和历史动作摘要。系统 prompt 还包含恢复策略：若页面被弹窗、cookie banner、登录浮层或验证码遮挡，优先选择安全关闭、Escape、等待或 `ask_user`；若最近动作没有推进页面状态，不要重复同一动作。

Algorithm 1: 单任务多步运行流程

Input: 起始 URL u_0，任务指令 I，最大步数 T，模型客户端 M
Output: run summary, trace artifacts, final answer

```
打开浏览器并访问 u_0
初始化历史动作、任务 checklist 和 trace 目录
for t = 0 to T − 1 do
    截图，提取候选元素、页面文本和 Accessibility Tree
    绘制 SoM 标注图并保存当前 step artifacts
    构造 ReAct prompt，调用模型 M 生成一个 JSON 动作
    解析动作；若非法 JSON，则保存原始响应并重试一次
    校验动作 schema、安全规则和 target_id
    执行动作并保存 execution_result.json
    调用 Progress Checker 判断是否推进页面状态
    if 动作为 answer 或 done then
        调用 Final Verifier 检查答案证据
        if 验证通过 then
            写入成功 summary 并结束
        end
    end
    if 连续卡住或达到恢复条件 then
        根据 checklist 生成提示、继续、询问用户或失败退出
    end
end
若超出最大步数，生成部分结果并保存失败类型
```

3.6 Progress Checker 与 Final Verifier

Progress Checker 通过 URL、候选元素数量、候选文本签名、截图 hash 等信号判断动作是否改变页面；同时检测重复动作、搜索/导航循环、URL ping-pong、导航参数被网站拒绝等情况。当动作连续没有改变有用状态时，系统会把恢复提示加入下一步 prompt，或在可配置情况下请求用户帮助。

Final Verifier 用于避免模型“自信地宣布完成”。当模型输出 `answer` 时，Verifier 会检查答案文本、URL、页面文本和历史证据是否匹配任务关键词；对中文编码可能受 API 代理影响的场景，还实现了 ASCII/数字片段回退匹配。当模型输出 `done` 时，Verifier 会检查近期是否有成功执行动作、任务中的字面量是否出现在证据中。CSRankings 等确定性任务还会在运行结束后由 `eval/validators.py` 再做答案级校验。

3.7 Trace 格式与可视化

每个 run 保存到 `traces/task_run/{run_id}/` 或用户指定目录。每一步的目录结构如下：

表 7 单步 trace 文件

文件	含义
`screenshot.png`	原始页面截图
`marked_screenshot.png`	带候选元素编号的 SoM 标注图
`candidates.json`	当前可交互候选元素列表
`accessibility_tree.txt`	当前页面可访问性树
`page_text.txt`	渲染文本与候选文本片段
`prompt.txt`	发送给模型的文本 prompt
`raw_response.txt`	模型原始输出，保留非法 JSON 与重试信息
`parsed_action.json`	解析后的结构化动作
`execution_result.json`	执行是否成功、执行前后 URL、错误类型和异常信息
`progress_check.json`	页面是否推进、是否重复/卡住、进展信号
`final_verification.json`	终止动作的证据检查结果

批量评测完成后，`eval/report_generator.py` 会生成 `summary.csv`、`summary.md` 和 `trace_report.html`。HTML 报告把每个 run 展开为 step-by-step 轨迹，包含截图、标注图、动作、执行结果、进展检查和最终验证，满足课程要求的“可视化操作轨迹（截图序列 + 动作序列）”。

4 任务设计与评测方案

4.1 任务库设计原则

课程要求每个网站至少 3 个任务。我们没有只选择简单搜索任务，而是按 easy、medium、hard 三档构建任务库，使任务难度逐步增加：easy 侧重页面定位和单点信息抽取；medium 需要跨页面查找、比较和结构化字段补全；hard 强调长链推理、跨来源验证、候选集合筛选和最终报告生成。

最终主线保留 CSRankings、Bilibili 和广州旅游三个 YAML 任务文件。每个任务统一定义任务标识、网站、难度、起始地址、自然语言说明、最大步数、成功标准和禁止动作。

表 8 最终 9 个评测任务

场景	难度	task id 摘要	任务摘要
CSRankings	easy	NLP professor	在 2020–2025 年美国 NLP 排名最高学校中寻找论文数最多、排名最靠前的教授，确定性答案为 Graham Neubig。
CSRankings	medium	CN ML scholar	找 2015–2025 年中国大陆高校 Machine Learning 排名最高学校中排名最高学者，并补全职位、邮箱、引用量、H-index 和最高引论文标题。
CSRankings	hard	AI/Sys. faculty	比较美国高校在 8 个 AI/Systems 方向前 30 名中的均衡性，计算中位数、平均值和最差排名，再找覆盖方向最广的 faculty。
Bilibili	easy	public video	从 B 站主页进入公开视频页，返回标题、链接和可见简介；若无简介需明确说明。
Bilibili	medium	AI top 3	打开“人工智能”搜索结果页，读取综合排序前三个 UP 主视频的可见信息，并分析受众与理由。
Bilibili	hard	bangumi report	浏览至少 5 个热门原创番剧详情，选 3 个外部站核验，最终选择 1 个推荐对象，获取相关视频下载链接并生成 HTML 展示页。
广州旅游	easy	top attraction	在携程查找广州景点页，返回排序第 1 景点的名称、评分、地址或区域、开放时间或门票提示和详情页 URL。
广州旅游	medium	Chimelong	只使用携程景点页，至少打开 3 个广州长隆相关景点详情，比较评分、地址、开放时间、票务提示并推荐 1 个。
广州旅游	hard	two-day plan	从广州南站进出，基于真实网页规划两天一晚行程；查酒店、餐厅、官方景点页面，给出交通、预算、雨天替换和来源 URL。

4.2 难度划分

难度不单由步数决定，而由“状态空间、证据要求、判分复杂度”共同决定。CSRankings easy 通常只需展开一个学校列表并读取教师信息；hard 则需要比较 8 个方向的学校排名、计算中位数和平均值，再对 faculty 的 # Pubs 做覆盖统计。Bilibili 与旅游任务更开放，hard 任务不仅需要浏览多个候选页面，还要跨来源核验和生成最终展示材料。

表 9 Easy/Medium/Hard 分层标准

难度	典型能力	评测重点
Easy	目标明确，少量页面跳转，单点信息抽取	元素定位、滚动、基础阅读、答案是否精确。
Medium	多页面跳转、字段补全、候选比较	跨页面记忆、来源记录、结构化输出完整性。
Hard	长链任务、跨来源验证、复杂计算或报告生成	计划分解、证据覆盖、避免循环、开放结果可复核性。

4.3 评测指标与判分口径

批量运行后，`eval/metrics.py` 从每个 `run_summary.json` 和 step trace 构造 run record，并计算以下指标：

- **Task Success Rate (SR)**：harness 判定成功的 run 数 / 总 run 数，反映主循环是否顺利停止并生成答案；
- **Validated Success Rate**：对确定性任务按答案规则复核后的成功率；
- **Average Steps / Average Time**：平均动作步数与耗时；
- **Action JSON Validity Rate**：解析成功且未使用 fallback 的动作步数占比；
- **Grounding Error Rate**：因 `target_id` 不存在、不可见或不可用导致的定位错误占比；
- **Correction/Retry Success Rate**：触发重试或恢复机制后最终成功的比例；
- **Failure Type Distribution**：将失败归类为 perception、planning、execution、verification、website 五类。

CSRankings 三题属于封闭答案任务，可由脚本严格复核。Bilibili 与广州旅游规划任务具有开放性，页面内容会随时间、推荐系统、地区和登录状态变化，因此先自动检查字段覆盖和来源 URL，再结合 trace 进行人工或 LLM-as-judge 复核。本文明确区分 **harness success**（流程正常结束）和 **validator success**（答案通过严格规则）；二者不能混用。

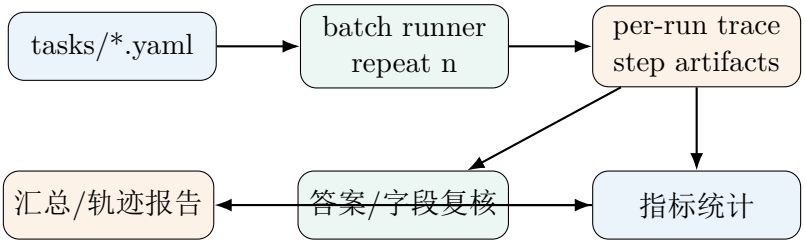


图 4 批量评测与报告生成流程

4.4 第一轮评测：DeepSeek V4 Pro 九任务重复评测

我们首先使用 DeepSeek V4 Pro 进行重复评测，作为本项目最主要的课程验收数据。实验按照“每个任务至少 10 次独立测试”的要求组织：CSRankings、Bilibili、广州旅游规划三个场景各包含 easy、medium、hard 三个任务，共 9 个任务；每个任务运行 10 次，总计 90 次。本小节的全部统计数值以项目随附的《实验报告》PDF 为统一来源，不再对分散的运行目录逐项重算。按该实验报告的统计口径，90 次中 76 次计为成功，综合成功率为 **84.44%**；平均步数为 12.7611，平均用时为 439.5247 秒；失败 14 次，其中 planning failure 13 次、perception failure 1 次。

表 10 DeepSeek V4 Pro 正式评测场景级结果

场景	运行次数	成功次数	成功率	说明
CSRankings	30	20	66.67%	easy 与 medium 均成功，hard 任务 10 次均失败。
广州旅游规划	30	26	86.67%	medium 失败 3 次，hard 失败 1 次。
Bilibili	30	30	100.00%	三个难度均计为成功；源报告将 hard 中 6 次标记为“参考补齐”。
合计	90	76	84.44%	失败总数 14 次。

表 11 DeepSeek V4 Pro 正式评测任务级结果

场景/难度	任务	次数	成功	成功率	平均步数	备注
CS easy	NLP top professor	10	10	100%	3.4	平均用时 186.19 秒。
CS medium	CN ML scholar profile	10	10	100%	8.1	平均用时 308.29 秒。
CS hard	AI/Systems balanced faculty	10	0	0%	22.4	规划失败 10 次,9 次卡住，1 次请求人工输入。
GZ easy	Ctrip top attraction	10	10	100%	5.1	平均用时 235.94 秒。
GZ medium	Chimelong sights comparison	10	7	70%	10.7	规划失败 3 次。
GZ hard	two-day real web plan	10	9	90%	34.1	感知失败 1 次,平均用时 1119.67 秒。
Bili easy	public video intro	10	10	100%	2.2	平均用时 235.37 秒。
Bili medium	AI search audience	10	10	100%	2.6	平均用时 287.21 秒。
Bili hard	bangumi HTML recommendation	10	10	100%	26.25	平均用时 708.60 秒。

从 DeepSeek V4 Pro 的结果可以看到三类模式。第一，CSRankings easy/medium、广州 easy 和 Bilibili 三个任务都达到 100%，说明在页面可访问且目标清晰时，文本观察空间已经足以支撑稳定的信息抽取和结构化回答。第二，CSRankings hard 的 0% 是最明显瓶颈：该任务要求跨 8 个方向收集排名、筛选学校、计算统计量，并继续比较 faculty 覆盖方向数，失败主要来自长链规划和中间证据组织不足。第三，开放式长任务并不一定失败，但成本显著更高，例如广州 hard 平均 34.1 步、1119.67 秒，Bilibili hard 平均 26.25 步、708.60 秒，说明长任务评测必须同时报告成功率、步数、耗时和失败类型。

4.5 GPT-5.5 九任务基线评测

在系统功能稳定后，我们使用 `gpt-5.5` 以 vision observation mode 对 9 个正式任务各运行一次，结果保存在 `reports/gpt55_9tasks_final_20260617/`。从主流程角度看，9 个任务均成功到达 `answer` 或 `done` 状态，**harness success 为 9/9**，平均步数为 10.89 步，平均用时约 721.10 秒，且没有出现 grounding error。这说明在网页可访问、视觉输入和上下文预算正常的条件下，强模型能够完成三类网站任务；其中 Bilibili hard 与广州 hard 也完成了多页面证据收集和最终结构化输出。

不过，严格复核不能只看 harness success。CSRankings hard 要求最终 faculty 为

Zhihao Jia，且 8 个方向排名、# Pubs 序列、覆盖方向数和总发表数均需匹配。该次 GPT-5.5 虽完成主流程，但 deterministic validator 得分为 0.5556，未通过严格答案校验。因此，本组结果应表述为：**流程层面 9/9 跑通；开放任务字段覆盖均为 1.0；确定性任务中 easy 与 medium 严格正确，hard 出现最终实体选择错误**。

表 12 GPT-5.5 九任务基线结果

任务	难度	harness	步数	复核结论
CSRankings NLP professor	easy	成功	3	validator 通过，答案为 Graham Neubig。
CSRankings ML profile	medium	成功	15	validator 通过，朱军档案字段匹配。
CSRankings AI/Sys. faculty	hard	成功	20	validator 未通过，长链统计中的最终选择错误。
Bilibili public video intro	easy	成功	2	字段覆盖 1.0，需结合 trace 复核具体视频页。
Bilibili AI search top 3 audience	medium	成功	1	字段覆盖 1.0，从搜索页可见信息完成分析。
Bilibili bangumi HTML	hard	成功	20	字段覆盖 1.0，完成候选浏览、外部核验和 HTML 输出。
Ctrip top attraction	easy	成功	4	字段覆盖 1.0，返回景点、评分、地址与详情页。
Ctrip Chimelong compare	medium	成功	6	字段覆盖 1.0，完成多景点详情页对比。
Guangzhou two-day plan	hard	成功	27	字段覆盖 1.0，完成酒店、餐饮、景点与路线规划。

4.6 十模型横向评测：先看有效样本

我们随后选取 10 个多样化模型进行横向评测，覆盖 OpenAI、Google Gemini、Anthropic Claude、Qwen、DeepSeek、Kimi、GLM 和豆包系列。视觉模型为 GPT-4o mini、GPT-5 mini、Gemini 2.5 Flash、Claude Haiku 4.5、Qwen3-VL-32B 与豆包 Vision Pro；文本模型为 DeepSeek-V4-Pro、Qwen3.7-Max、Kimi-K2.6 与 GLM-5.2。完整 API 名称见表 13。前 6 个按 vision mode 运行，后 4 个按 text mode 运行。9 个任务与 10 个模型构成 90 个计划的模型–任务组合；为避免云端浏览器对 Google Scholar 的不稳定访问影响实验，CSRankings 三题在本地运行，Bilibili 与广州任务在云服务器运行。

为了避免把中转站超时、内容安全拦截、上下文超限等服务层问题误计为模型能力失败，我们把每个组合标记为 **valid**、**invalid** 或 **incomplete**。表 13 先汇报可正常取得完整 trace 的有效样本："CS（有效/严格成功）" 的分母为 3 个 CSRankings 任务；"开放（有效/完成）" 的分母为 Bilibili 与广州的 6 个任务，完成数表示已完成主流程并通过轨迹初筛，仍不替代最终人工复核。

表 13 十模型九任务横向评测：有效样本结果

模型	模式	CS 严格成功	开放完成	无效	未完成	有效样本中的主要观察
gpt-4o-mini	vision	3/0	6/2	0	0	CS hard 与开放 medium/hard 易进入动作循环，规划稳定性不足。
gpt-5-mini	vision	3/2	6/4	0	0	easy/medium 较稳；CS hard 与广州 hard 仍暴露长链统计和多源规划问题。
gemini-2.5-flash	vision	3/1	6/3	0	0	可完成部分阅读任务，但在广州 medium/hard 与 B 站 hard 中出现证据收集循环。
claude-haiku-4-5	vision	3/1	0/0	6	0	开放任务输入超限，不能直接纳入开放任务能力比较；CS hard 未过严格校验。
qwen3-vl-32b	vision	3/2	6/4	0	0	CS easy/medium 与 B 站任务较好，广州 medium/hard 的真实网页导航仍不稳定。
doubao-vision-pro	vision	0/0	0/0	9	0	图文总 token 超限，当前配置下无可比较有效样本。

模型	模式	CS 严格成功	开放完成	无效	未完成	有效样本中的主要观察
DeepSeek-V4-Pro	text	3/2	6/5	0	0	纯文本模式表现稳健，说明 DOM/A11y/page text 对信息型任务有较强支撑。
qwen3.7-max	text	2/2	5/5	2	0	有效样本均完成，但广州 hard 出现内容安全拦截，需要补跑。
kimi-k2.6	text	2/2	3/2	2	2	有效样本尚可，但服务超时和长任务未完成影响正式统计。
GLM-5.2	text	2/2	4/3	3	0	easy 与 B 站 medium 较好；广州 medium 超时、广州 hard 被内容拦截。

从有效样本可得出三点结论。第一，**确定性答案任务比开放任务更能暴露“答对没有”：**CSRankings easy/medium 对强文本模型和部分视觉模型并不困难，但 hard 要求稳定完成 8 个方向排名统计和 faculty 覆盖度比较，多个模型出现了流程完成但最终实体或字段不匹配的问题。第二，**视觉输入并不必然带来更高成功率：**在当前 observation 较重的设置下，视觉模型还要承担截图和 SoM 图像成本；而 DeepSeek-V4-Pro、qwen3.7-max 等文本模型依靠 DOM candidates、Accessibility Tree 和页面文本，在若干信息抽取任务中取得了稳定结果。第三，**开放长链任务的主要失败不是 JSON 格式错误，而是规划和证据管理：**典型失败是重复点击同一入口、在搜索页和详情页之间循环，或收集到局部证据后未及时转向下一子目标，说明 checklist 和历史证据回灌仍需进一步结构化为可检索 notebook。

4.7 十模型评测中的无效样本与风险

在报告有效样本结论之后，还必须单独说明不可归因为模型能力的技术问题。第一类是 **API read timeout**，如 `kimi-k2.6` 的 Bilibili medium、`GLM-5.2` 的广州 medium：请求已发出，但中转站或上游模型在超时时间内没有返回，应标为 `invalid_api_timeout` 并补跑。第二类是 **HTTP 400 内容安全拦截**，如 `qwen3.7-max` 和 `GLM-5.2` 在广州 hard 中被上游安全策略拦截；真实网页文本中下载链接、支付、登录、预约等敏感词可能共同触发代理侧策略，不能直接解释为规划能力差。第三类是 **上下文或图文 token 超限：**`claude-haiku-4-5` 的 6 个开放任务、`doubao-1-5-vision-pro-32k` 的多个任务因此无效。

造成 token 超限的直接原因是当前 vision mode 的 observation 较重：每一步会同时发送任务、动作历史、候选元素、页面文本、Accessibility Tree、raw screenshot 与 marked screenshot。以 Bilibili easy 为例，Claude 的第一步 prompt 文本约 55 KB、Accessibility

Tree 约 36 KB、候选元素约 25 KB，两张截图接近 1.9 MB；经 base64 和代理侧估算后约为 428k tokens，超过上下文限制。后续正式横向评测应采用轻量 observation：只发送一张压缩 marked screenshot，候选元素仅保留可见高价值项，限制 A11y 长度，并按模型设置上下文预算。只有完成此类适配后，跨模型成功率才适合直接做排行榜比较。

4.8 数据来源与可视化轨迹

DeepSeek V4 Pro 第一轮 90 次重复评测的场景级、任务级和总体统计均以项目随附的《实验报告》PDF 为准；GPT-5.5 的单模型结果来自 `reports/gpt55_9tasks_final_20260617/summary.csv`；十模型汇总及每模型 trace 位于 `reports/model_suite/` 下的实验目录。后两组实验的每个 run 都保留 step 级截图、候选元素、prompt、模型响应、结构化动作、执行结果、进度检查和最终验证；HTML trace report 进一步按步骤展示截图、动作、模型 thought、执行结果与最终答案。它们既是开放任务人工复核的依据，也是定位感知、规划、执行和网站环境错误的证据。

5 错误分析、局限与展望

5.1 从有效样本看失败模式

第一轮 DeepSeek V4 Pro 重复评测首先暴露出一个关键现象：同一个模型在 easy/medium 任务上可以非常稳定，但在 hard 任务上会系统性失败。CSRankings hard 需要依次完成 8 个方向的学校排名统计、候选学校筛选、并列规则计算，再回到 faculty 层统计 # Pubs 覆盖；按《实验报告》的 10 次统计，该任务均未通过，主要表现为重复筛选、证据不足、卡住或请求人工确认。GPT-5.5 和十模型横向实验也出现类似现象：模型可能已浏览大量页面、甚至结束主流程，但仍选择错误 faculty 或遗漏关键字段。此类样本应归为规划或验证失败，而不应因 harness success 被记为成功。

开放任务中，Bilibili 和广州旅游的 medium/hard 更强调跨页证据管理。DeepSeek V4 Pro 在 Bilibili 三个任务上均计为 10/10 成功，在广州 hard 上 9/10 成功，但广州 hard 平均步数达到 34.10，说明开放长链任务即使能完成，也会显著放大证据组织成本。十模型实验中的典型失败包括在搜索页与详情页间重复跳转、只读到部分候选项就提前回答、或收集了信息却没有记录其来源 URL。这说明 JSON 动作合法并不保证任务真的完成，进度检查、任务清单和 trace 证据共同构成了评测闭环。

5.2 失败类型与改进方向

表 14 失败类型、典型表现与改进方法

类型	典型表现	可能改进
感知失败	看不到弹窗、误读截图、候选元素缺失或编号遮挡	改进元素过滤、A11y 补充、截图裁剪、多尺度 SoM；对图像密集页面加入 OCR。
定位失败	选择不存在或不可见的 `target_id`，点击后无响应	更严格候选生成，失败后自动重新观察，将定位错误反馈给模型。
规划失败	重复搜索、遗漏任务字段、过早回答、长链任务忘记证据	引入 checklist、任务分解、结构化 notebook，将 planner 与 executor 分离。
执行失败	网站跳转异常、按钮被遮挡、网络超时、页面加载慢	更稳定的等待策略、弹窗处理、重试机制和环境错误分类。
验证失败	harness 成功但答案错误，开放任务字段覆盖不等于真实完成	确定性 validator、LLM-as-judge、人工复核和 trace 证据约束。
网站/服务失败	站点访问异常、API timeout、HTTP 400、安全策略或上下文上限	与模型能力错误分离记录，保留原始报错并在环境恢复后补跑。

本轮数据中，DeepSeek V4 Pro 正式评测 90 次成功 76 次，失败 14 次；失败类型主要集中在 planning failure 13 次，另有 perception failure 1 次。十模型有效样本也没有出现大量结构化动作解析失败。因此，瓶颈主要转移到长链任务的计划分解、证据组织和最终严格校验。另一方面，API timeout、内容拦截和图文 token 超限说明代理服务、模型接口和 observation 预算同样会决定评测是否有效。因此，报告结果时应同时呈现有效样本结论、无效样本数量及其原因，而不能把所有失败压缩为单一成功率。

5.3 当前局限

当前系统已经具备课程项目所需的完整闭环，但仍有明显局限。

1. **开放任务自动评测仍偏弱。**字段覆盖和 URL 数量检查只能判断答案形式，无法完全判断证据真实性和推荐合理性；Bilibili hard 与旅游 hard 仍需人工或更强 LLM judge 结合 trace 复核。
2. **跨模型输入尚未完全归一。**视觉模型需要处理截图与 SoM，文本模型只接收 DOM、A11y 和页面文本；这符合各自接口能力，但 token 预算与输入信息量仍会影响横向可比性。
3. **真实网站不可控。** B 站推荐、携程页面和搜索结果随时间、地区、登录状态和反爬策略变化，导致重复运行的可比性下降。
4. **复杂页面操作仍不稳定。**筛选器、下拉框、悬浮弹窗和懒加载列表会使候选元素提取不完整；模型有时会依赖猜测 URL 参数，而不是逐步操作可见控件。

5. **复杂报告生成可能放大错误。**当最终答案含有未核实事实时，HTML report 会使结果显得更完整，因此必须在生成前保留严格证据约束。

5.4 后续工作

第一，评测更严格。对 CSRankings 类封闭任务继续维护确定性 validator；对 Bilibili 和旅游规划任务，引入同时读取最终答案、trace 中访问 URL、截图序列和任务标准的 LLM-as-judge，并报告自动层与复核层两套结果。

第二，感知更稳定、更轻量。优化候选元素过滤和 SoM 布局，区分 viewport 内、屏幕外可滚动及弹窗元素；对视觉模型采用压缩单图和高价值元素摘要，对纯文本模型清洗 A11y 与页面文本，从而降低上下文超限风险。

第三，规划更可控。将复杂任务拆为 planner 和 executor 两层：planner 先生成 checklist 与证据需求，executor 逐项收集证据；每完成一个字段就写入带 URL 和截图引用的结构化 notebook，最终答案仅引用已核验事实。

5.5 总结

Web-MMAgent 已从最小浏览器闭环扩展为可运行、可评测、可回放的网页多模态 Agent 系统。它将网页截图、DOM 候选元素、Accessibility Tree、SoM 标注图和页面文本组合成混合观察空间，用结构化 JSON 动作和安全校验约束模型行为，并通过 Progress Checker、Final Verifier、YAML 任务管理、批量评测和 HTML trace report 支撑课程要求的成功率统计、失败分类和可视化轨迹。

DeepSeek V4 Pro 重复评测说明，当前系统已经能支持同一模型在真实网页上的多次独立运行和失败类型统计，但 hard 任务会稳定暴露长链规划瓶颈；GPT-5.5 基线说明强模型可以完成九任务主流程，但 CSRankings hard 的严格失败也证明了验证层不可省略；十模型实验进一步显示，信息抽取任务可由文本或视觉 observation 支撑，而开放长链任务的关键瓶颈仍是规划、证据管理和环境稳定性。未来应以更轻量的 observation、可检索 notebook 和双层评测口径为基础，继续完善该课程级 Web Agent 实验平台。

参考文献

[1] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems 36*, 2023.

[2] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[3] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905. Association for Computational Linguistics, 2024.

[4] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890. Association for Computational Linguistics, 2024.

[5] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The BrowserGym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2024.

[6] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61349–61385. PMLR, 2024.

[7] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[9] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

[10] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. Windows Agent Arena: Evaluating multi-modal OS agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.

[11] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.

[12] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[13] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203*, 2024.

[14] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332. Association for Computational Linguistics, 2024.

[15] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. WebLINX: Real-world website navigation with multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 33007–33056. PMLR, 2024.